

Documentazione ExpSplit



PROGETTO DI PROGETTAZIONE, ALGORITMI E
COMPUTABILITÀ - 38090-MOD1

FABIO PEDRINI - 1042015

UNIVERSITÀ DEGLI STUDI DI BERGAMO

2025 - 2026

Indice

1	Iterazione 0	4
1.1	Introduzione	4
1.2	Requisiti	4
1.2.1	Requisiti Funzionali (RF)	4
1.2.2	Requisiti Non Funzionali (RNF)	5
1.3	Casi d'uso	5
1.4	Architettura	6
1.4.1	Architettura esagonale	6
1.4.2	Architettura a microservizi	7
1.5	Toolchain e tecnologie	7
2	Iterazione 1	9
2.1	Introduzione	9
2.2	Deployment	9
2.3	Architettura del servizio di autenticazione	11
2.4	Architettura del servizio di gateway	12
2.5	Casi d'uso	13
2.5.1	UC1.1 - Registrazione	13
2.5.2	UC1.2 - Autenticazione	13
3	Iterazione 2	15
3.1	Introduzione	15
3.2	Architettura del servizio ExpSplit	15
3.2.1	UserProfile	15
3.2.2	Group	16
3.2.3	Transaction e Settlement	18
3.2.4	Balance e Statistics	19
3.2.5	Strategia di calcolo	20
3.3	Diagrammi di sequenza	21
3.4	Casi d'uso	23
3.4.1	UC3.1 - Creazione gruppo	23

3.4.2	UC3.2 - Invito	24
3.4.3	UC4.1 - Creazione transazione	25
3.4.4	UC5.1 - Aggiornamento (bilancio di gruppo)	26
3.4.5	Altri casi d'uso	26
3.5	Modello REST	27
3.6	Testing	28
4	Iterazione 3	30
4.1	Introduzione	30
4.2	Deployment con Docker	30
4.2.1	Rete	32
4.2.2	Configurazione e inizializzazione	32
4.3	Sicurezza	32
4.4	Nuovo caso d'uso	32
4.4.1	UC3.5 - Elenco inviti in sospeso	32
4.5	Arricchimento delle risposte REST	33
4.6	Testing	34
5	Iterazione 4	35
5.1	Introduzione	35
5.2	Ruolo nell'architettura esagonale	35
5.3	Organizzazione del client	35
5.4	Funzionalità	36
5.5	Verifica e limiti	36

Elenco delle figure

1	Deployment	10
2	Sequenza di autenticazione (UC1.2)	12
3	Diagramma delle classi (Group)	17
4	Diagramma delle classi di Transaction e Settlement	18
5	Diagramma delle classi di Balance e Statistics	19
6	Creazione di una transazione con verifica di appartenenza al gruppo)	22
7	Richiesta del bilancio di un gruppo	23
8	Risultati test	29
9	Deployment con Docker Compose e mTLS interno	31

Elenco delle tabelle

1	Endpoint REST del servizio ExpSplit	28
2	Nuovo endpoint: invitations	33

1 Iterazione 0

1.1 Introduzione

ExpSplit è un servizio progettato per semplificare la gestione delle spese condivise tra gruppi di utenti. Il sistema permette il tracciamento e la divisione automatica e trasparente delle spese in contesti di gruppo.

Il servizio consente quindi di:

- Creare e gestire gruppi privati di spesa;
- Registrare spese condivise e rimborsi;
- Calcolare automaticamente la ripartizione delle spese;
- Fornire statistiche e un riepilogo delle spese del gruppo.

1.2 Requisiti

Il servizio deve soddisfare i seguenti requisiti funzionali e non funzionali:

1.2.1 Requisiti Funzionali (RF)

- **Gestione utente:** il servizio deve permettere a un utente di registrarsi e autenticarsi. La gestione utente deve quindi includere la possibilità di modificare le proprie informazioni personali e di accesso;
- **Gestione gruppo:** è necessario un meccanismo di creazione di un nuovo gruppo o di accesso a uno preesistente con relativi permessi;
- **Registro transazioni:** ogni utente all'interno di un gruppo deve avere la possibilità di creare, visualizzare e modificare le transazioni legate a spese o rimborsi tra i membri del gruppo;
- **Bilanci:** il servizio deve prevedere la possibilità di mostrare un bilancio sempre aggiornato per gruppo e per utente sulla base del registro transazioni;
- **Statistiche:** le transazioni possono essere categorizzate e tra diversi utenti in un gruppo. Si vuole prevedere un meccanismo di generazione delle statistiche sulla base dei dati a disposizione.

1.2.2 Requisiti Non Funzionali (RNF)

- **Sicurezza:** tutte le funzionalità, le risorse salvate e i vincoli di accesso devono essere rispettati e protetti;
- **Efficienza:** il servizio deve poter gestire molteplici aggiornamenti dei dati senza subire rallentamenti o blocchi;
- **Modularità:** il servizio deve essere progettato per l'aggiunta o rimozione di elementi che permettono di estendere o adattare le funzioni principali;
- **Scalabilità:** il servizio deve essere configurato per un contesto variabile su più sistemi indipendenti.

1.3 Casi d'uso

I casi d'uso sono riportati di seguito:

- **UC1 - Autenticazione**
 - **UC1.1 - Registrazione:** un utente può creare un account;
 - **UC1.2 - Autenticazione:** un utente registrato può eseguire l'accesso alle funzioni del servizio.
- **UC2 - Utente**
 - **UC2.1 - Modifica:** un utente può inserire informazioni personali e aggiuntive;
 - **UC2.2 - Eliminazione:** un utente può decidere di eliminare permanentemente il proprio profilo.
- **UC3 - Gestione gruppo**
 - **UC3.1 - Creazione:** un utente può creare un gruppo di spesa;
 - **UC3.2 - Invito:** un utente può invitare un altro utente in un gruppo. L'invito può essere accettato o rifiutato;
 - **UC3.3 - Abbandono:** un utente può uscire da un gruppo;
 - **UC3.4 - Eliminazione:** un utente può eliminare un gruppo.
- **UC4 - Transazioni**

- **UC4.1 - Creazione:** un utente può creare transazioni di diverso tipo con ripartizione della spesa personalizzata tra gli utenti dello stesso gruppo;
- **UC4.2 - Modifica:** un utente può modificare una transazione creata in precedenza all'interno di un gruppo;
- **UC4.3 - Cancellazione:** un utente può eliminare definitivamente una transazione creata in precedenza all'interno di un gruppo;
- **UC5 - Bilanci**
 - **UC5.1 - Aggiornamento:** un utente può richiedere un bilancio sempre aggiornato rispetto al registro delle transazioni di un gruppo;
 - **UC5.2 - Dettaglio:** un utente può richiedere un bilancio dettagliato per le transazioni associate al suo profilo verso il gruppo o verso altri utenti nel gruppo;
 - **UC5.3 - Statistiche:** un utente può richiedere informazioni aggregate sulla base delle informazioni addizionali disponibili nelle transazioni.

1.4 Architettura

L'architettura di *ExpSplit* è basata sul pattern **esagonale** (*ports and adapters*), applicato a livello di singolo servizio, e su una scomposizione a micro-servizi a livello di sistema.

1.4.1 Architettura esagonale

Ogni servizio è strutturato secondo il pattern esagonale (*ports and adapters*), con l'obiettivo di isolare la logica applicativa dai dettagli tecnologici e infrastrutturali. Si distinguono i seguenti elementi:

- **Domain:** contiene le entità e le regole di business, indipendenti da qualsiasi tecnologia o framework;
- **Casi d'uso:** implementano le funzionalità del sistema descritte in precedenza;
- **Porte:** interfacce (in ingresso e uscita) che definiscono i punti di scambio tra l'applicazione e il mondo esterno;

- **Adapter:** implementazioni concrete che collegano l'applicazione a tecnologie specifiche.

Grazie a questa scelta, gli *adapter* possono essere sostituiti o aggiunti senza modificare il *domain* e i *casi d'uso*.

1.4.2 Architettura a microservizi

A livello di sistema, *ExpSplit* è organizzato secondo un'architettura a **microservizi**, ciascuno progettato internamente secondo il pattern esagonale descritto sopra. I servizi individuati sono i seguenti:

- **Gateway:** punto di accesso unico al sistema, responsabile dell'instradamento delle richieste verso i servizi interni;
- **Auth:** servizio dedicato alla gestione dell'autenticazione e delle credenziali degli utenti;
- **ExpSplit:** servizio contenente la logica di dominio del sistema (gruppi, transazioni, bilanci, statistiche), corrispondente ai requisiti funzionali descritti in precedenza.

Questa scomposizione favorisce la *scalabilità* orizzontale dei singoli servizi in modo indipendente e la loro evoluzione autonoma, in linea con i requisiti non funzionali individuati.

1.5 Toolchain e tecnologie

Per lo sviluppo si prevede l'utilizzo dei seguenti strumenti e tecnologie:

- **IDE:** IntelliJ IDEA Ultimate per lo sviluppo del sistema e XCode per l'applicazione mobile.
- **Backend:** Spring Boot con i moduli Web, Security e ulteriori dipendenze per l'implementazione della sicurezza e dell'autenticazione.
- **Build tool:** Gradle per la gestione delle dipendenze e del ciclo di build.
- **Database:** PostgreSQL per la persistenza dei dati.

- **Database per i test:** H2 in-memory, per eseguire test unitari e di integrazione senza dipendere dal database di produzione.
- **Testing unitario:** JUnit per la verifica dei singoli componenti.
- **Testing di integrazione:** MockMvc combinato con JUnit per testare i flussi tra controller, servizi e layer di persistenza.
- **Frontend:** Applicazione demo sviluppata in Swift e SwiftUI.
- **Strumenti di test API:** Postman / curl.
- **Versionamento:** Git + GitHub per il controllo della versione del codice sorgente.
- **Deployment/Container:** Docker per creare ambienti coerenti tra sviluppo e produzione, e Docker Compose per orchestrare i microservizi (gateway, auth, expsplit) insieme al database in ambiente locale.

2 Iterazione 1

2.1 Introduzione

L'iterazione 1 ha l'obiettivo di realizzare il servizio di **autenticazione** e di **gateway**, definendo in modo dettagliato le scelte implementative e il modello di *deployment* previsto del sistema.

2.2 Deployment

Il sistema è composto da tre servizi indipendenti (*Gateway*, *Auth*, *ExpSplit*).

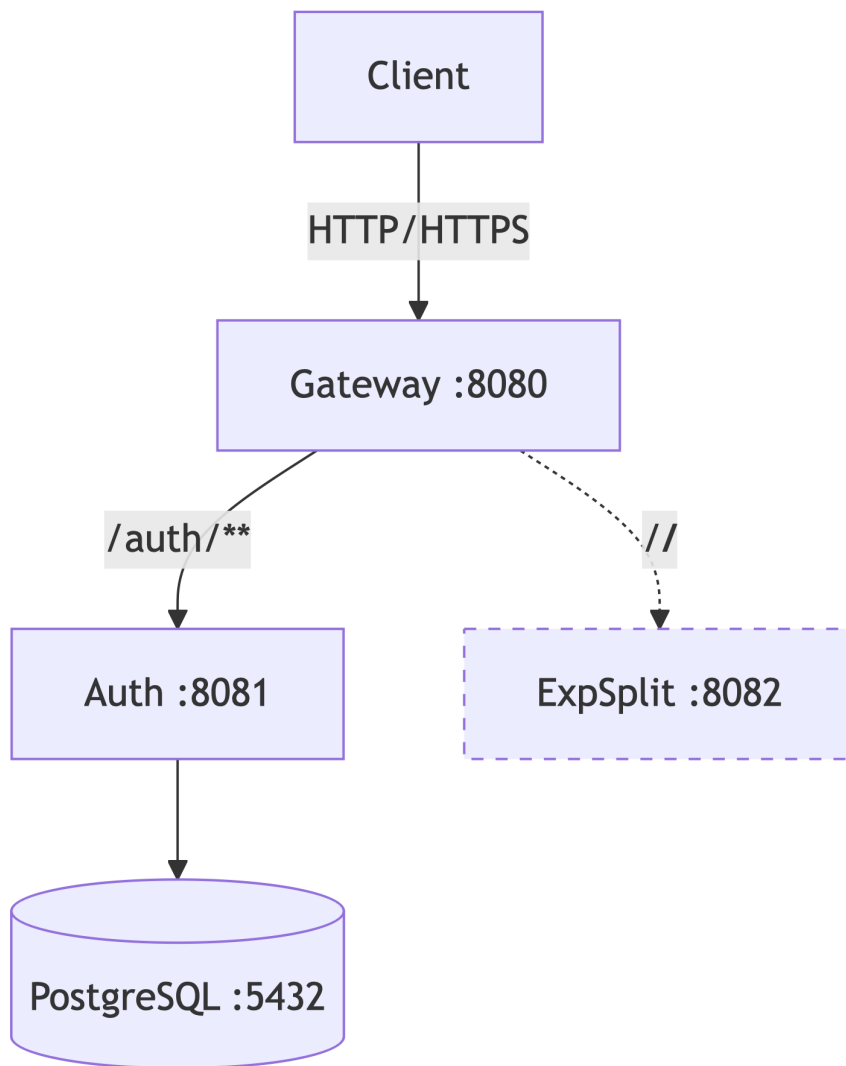


Figura 1: Deployment

La configurazione è impostabile tramite variabili d'ambiente. I servizi possono condividere un unico database e mantenere la separazione delle funzioni con i giusti accorgimenti sui ruoli e permessi delle tabelle. Questa parte verrà meglio definita con il servizio *ExpSplit*.

Ogni servizio può essere containerizzato ed eseguito in modo indipendente.

2.3 Architettura del servizio di autenticazione

Il servizio *Auth* segue il pattern esagonale applicato all'unico elemento che lo caratterizza: `UserAuth` (identità di accesso).

- **Dominio:** `UserAuth` (email, password con relativo algoritmo, stato `ACTIVE/LOCKED/DISABLED`) incapsula il blocco temporaneo dopo tentativi falliti ripetuti;
- **Casi d'uso:** `RegistrationService` e `AuthenticationService` implementano le porte in ingresso, dipendendo solo da porte in uscita;
- **Adapter in ingresso:** `AuthController` espone `/auth/register` e `/auth/login`; un *exception handler* centralizzato traduce le eccezioni di dominio in risposte HTTP;
- **Adapter in uscita:** persistenza JPA, hashing Argon2id delle password ed emissione di token JWT.

I token sono JWT firmati con RS256: la coppia di chiavi è generata e salvata al primo avvio e pubblicata secondo lo standard JWK Set (`/.well-known/jwks.json`). La chiave pubblica esposta agli altri servizi permette di non dover dipendere dal servizio *Auth* per la verifica del token.

La figura di seguito mostra il flusso di accesso da parte di un utente:

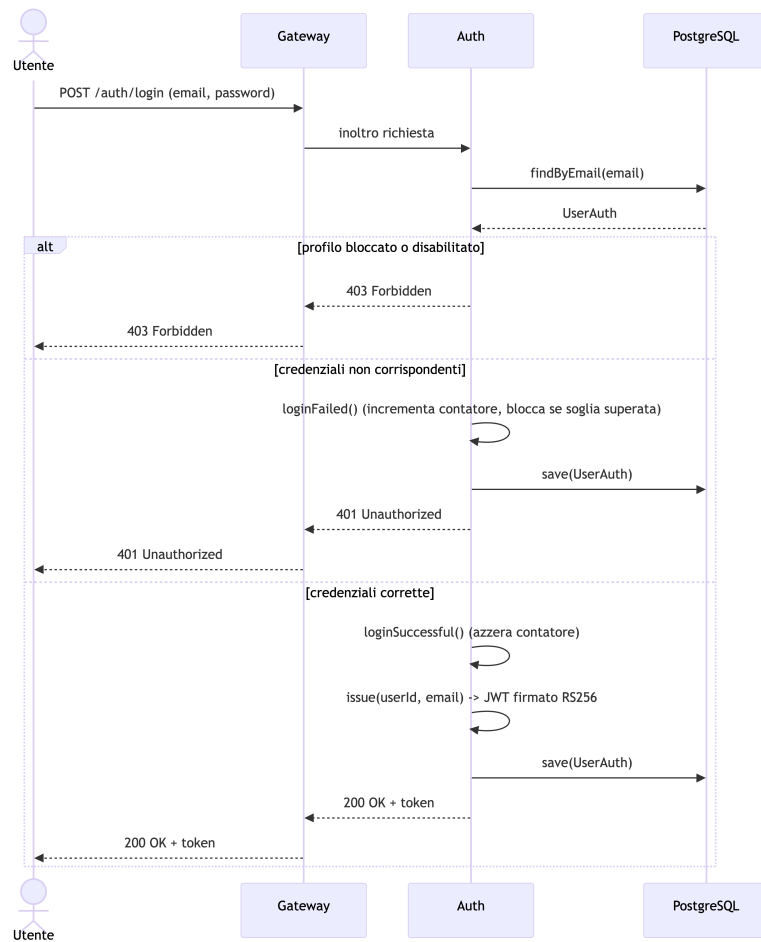


Figura 2: Sequenza di autenticazione (UC1.2)

La sessione è *stateless*. Sono pubblici solo registrazione, login e JWKS; la verifica dei token è compito del servizio *ExpSplit*.

2.4 Architettura del servizio di gateway

Il *Gateway* è l'unico punto di accesso esterno al sistema e svolge il ruolo di *reverse proxy* senza avere quindi particolari elementi che lo caratterizzano oltre alla sua funzione. Tutte le richieste verso `/auth/**` sono per il servizio *Auth* mentre le restanti per il servizio *ExpSplit*.

2.5 Casi d'uso

In questa iterazione viene realizzato esclusivamente **UC1 - Autenticazione**:

2.5.1 UC1.1 - Registrazione

- **Attori:** Utente non registrato;
- **Descrizione:** un utente fornisce un indirizzo email e una password per creare un nuovo profilo di autenticazione nel sistema;
- **Precondizioni:** l'indirizzo email fornito non deve essere già associato a un profilo esistente;
- **Postcondizioni:** viene creato un nuovo profilo utente in stato **ACTIVE**, con la password memorizzata (cifrata), e viene restituito l'identificativo univoco del profilo creato;
- **Flusso degli eventi principale:**
 1. l'utente invia una richiesta di registrazione (email, password) all'endpoint `POST /auth/register`;
 2. il servizio valida i dati ricevuti (email non vuota, password tra 8 e 128 caratteri);
 3. il servizio verifica l'unicità dell'e-mail;
 4. la password viene cifrata con l'algoritmo Argon2id;
 5. viene creato e salvato un nuovo profilo utente (**ACTIVE**);
- **Flussi alternativi:**
 - *Email già registrata:* l'operazione viene interrotta e viene restituito un errore (**409 Conflict**);
 - *Dati non validi:* email vuota o password fuori dai vincoli di lunghezza comportano un errore di validazione (**400 Bad Request**).

2.5.2 UC1.2 - Autenticazione

- **Attori:** Utente registrato;
- **Descrizione:** un utente registrato fornisce le proprie credenziali per ottenere un token di accesso da utilizzare nelle richieste successive verso il sistema;

- **Precondizioni:** l'utente deve possedere un profilo esistente, non bloccato o disabilitato;
- **Postcondizioni:** in caso di successo viene restituito un token firmato, valido per un periodo limitato, e il contatore dei tentativi falliti viene azzerato; in caso di fallimento il contatore dei tentativi falliti viene incrementato;
- **Flusso degli eventi principale:**
 1. l'utente invia una richiesta di autenticazione (email, password) all'endpoint `POST /auth/login`;
 2. il servizio recupera il profilo associato all'email fornita;
 3. il servizio verifica che il profilo non sia bloccato;
 4. il servizio verifica la corrispondenza tra la password fornita e quella memorizzata;
 5. in caso di corrispondenza, il contatore dei tentativi falliti viene azzerato e viene generato un token;
 6. il token viene restituito all'utente.
- **Flussi alternativi:**
 - *Credenziali non valide:* il contatore dei tentativi falliti viene incrementato; superata la soglia di tentativi consentiti il profilo viene bloccato temporaneamente; il sistema restituisce un errore (401 `Unauthorized`);
 - *Profilo bloccato o disabilitato:* la richiesta viene rifiutata senza verificare la password, restituendo un errore (403 `Forbidden`).

3 Iterazione 2

3.1 Introduzione

L'iterazione 2 realizza il servizio *ExpSplit*: tutti i casi d'uso vengono implementati e verificati.

A differenza del servizio *Auth*, caratterizzato unicamente da (**UserAuth**), *ExpSplit* è composto da più elementi distinti che collaborano tra loro.

3.2 Architettura del servizio ExpSplit

Il servizio applica il pattern esagonale a livello di ogni singolo *elemento* di dominio organizzato in un pacchetto dedicato con la struttura **domain / application / adapters** e possiede il proprio repository.

Le due funzionalità (**Balance** e **Statistics**) sono classi di appoggio non persistenti che leggono i dati di **Transaction** e **Settlement** già salvati e producono un risultato al momento della richiesta.

- **Riferimenti:** ogni elemento conosce esclusivamente l'identificativo dell'altro elemento (es. **Group** ha solo accesso a un **UserProfileId** dei propri membri, non l'oggetto **UserProfile**). Questa scelta permette di mantenere l'indipendenza rigida tra i modelli;
- **Dipendenza:** quando un'operazione richiede più elementi (es. verificare che un utente sia membro di un gruppo prima di creare una transazione), la verifica avviene nel *servizio applicativo*, che dipende dai repository di entrambi gli elementi coinvolti.
- **Validazione token:** la validazione del token JWT viene eseguita nel servizio *ExpSplit* (tramite chiave pubblica JWKS esposta da *Auth*). Ogni endpoint implementato in *ExpSplit* necessita di un token valido.

3.2.1 UserProfile

UserProfile rappresenta le informazioni personali di un utente all'interno di *ExpSplit* ed è volutamente disaccoppiato da **UserAuth**: il servizio *Auth* gestisce solo le credenziali, mentre *ExpSplit* gestisce le informazioni aggiuntive.

3.2.2 Group

Group rappresenta un gruppo di spesa condivisa. La relazione e i permessi di un gruppo e utente sono concretizzati in **Membership**. Tutte le operazioni eseguibili su **Membership** sono eseguibili solo tramite i metodi di **Group**.

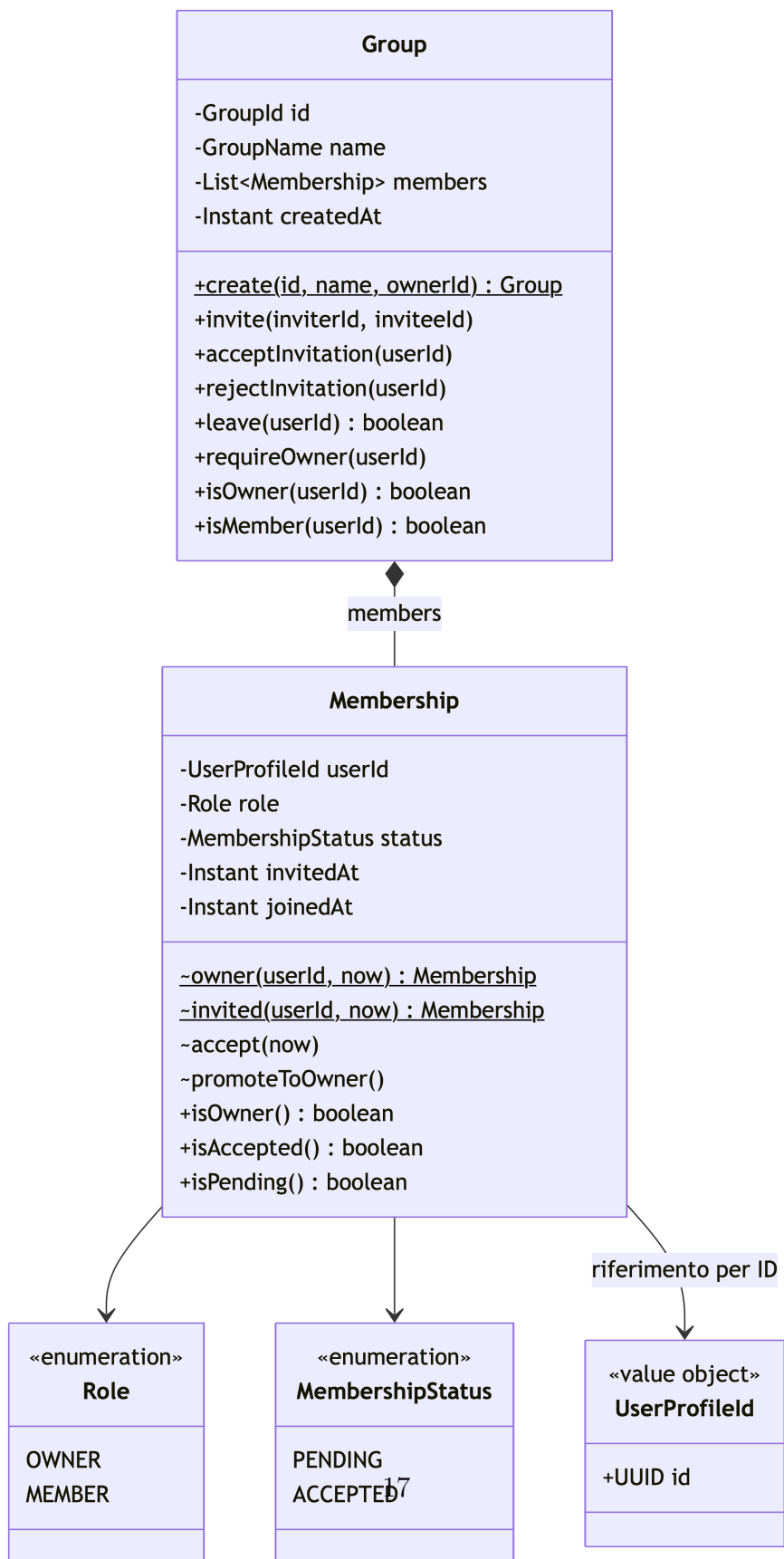


Figura 3: Diagramma delle classi (Group)

Per aggiungere un utente in un gruppo si utilizzano gli inviti. Un invito segue un ciclo a due fasi: un membro viene aggiunto in stato **PENDING** da un **Owner**, e transita a **ACCEPTED** solo quando l'invitato stesso accetta (o viene rimosso, se rifiuta). Se l'**Owner** abbandona il gruppo, il membro accettato più anziano (per data di adesione) viene promosso automaticamente a nuovo **Owner**; se non restano membri accettati, il gruppo viene eliminato automaticamente.

3.2.3 Transaction e Settlement

All'interno di un gruppo ci possono essere due tipi di movimenti finanziari: **Transaction** e **Settlement**.

- **Transaction** rappresenta una spesa condivisa: un pagatore anticipa un importo, ripartito tra i membri del gruppo tramite percentuali (**TransactionShare**);
- **Settlement** rappresenta un pagamento diretto tra due utenti per saldare un debito, senza alcuna ripartizione.

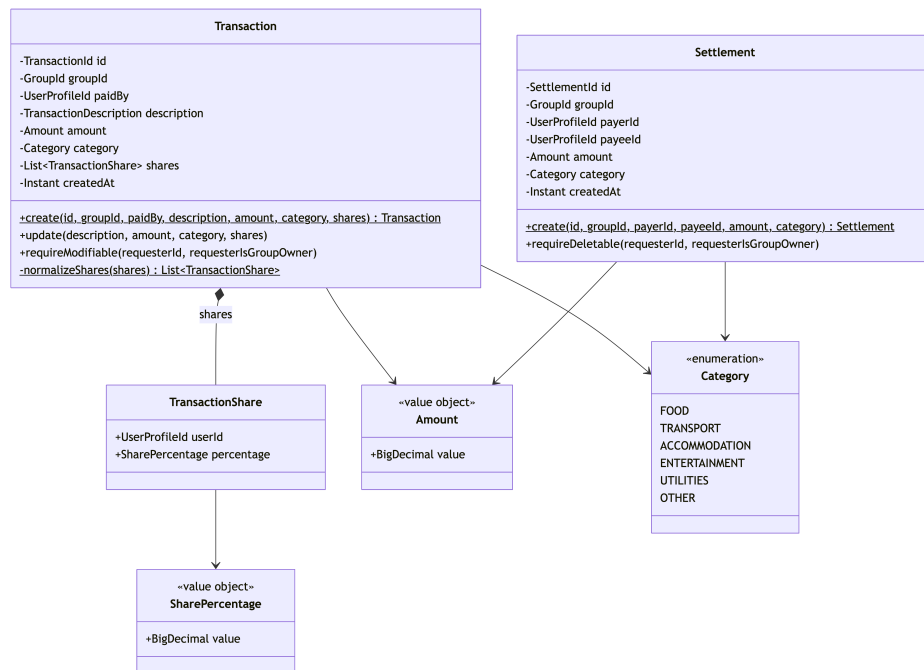


Figura 4: Diagramma delle classi di Transaction e Settlement

Le percentuali di ripartizione di una **Transaction** sono impostabili dall'utente e devono sommare a 100, a meno di qualche approssimazione che il servizio può gestire aggiungendo lo scarto all'ultima quota (fino a 0,01 per ogni quota). Scarti superiori a questa tolleranza vengono invece rifiutati come input non valido. Molti campi tra **Transaction** e **Settlement** sono simili, ma sono stati volutamente tenuti separati per la loro funzione ed eventuale estensione. La modifica e la cancellazione sono permesse solo a chi ha creato il movimento oppure dall'**Owner** del gruppo.

3.2.4 Balance e Statistics

I bilanci e le statistiche sono pure classi logiche che leggono le **Transaction** e i **Settlement** già esistenti di un gruppo e producono un risultato aggregato al momento della richiesta, senza alcuno stato salvato.

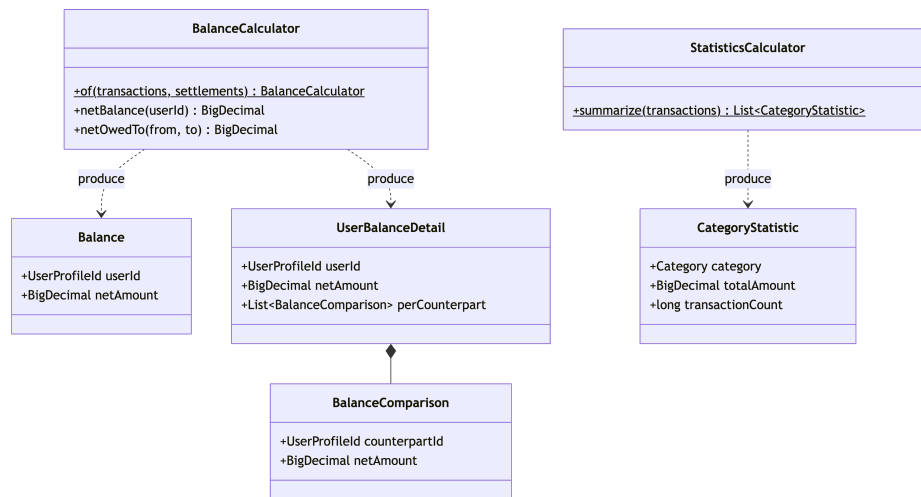


Figura 5: Diagramma delle classi di Balance e Statistics

BalanceCalculator costruisce, a partire dalle spese e dai pagamenti diretti di un gruppo, un registro di debiti a coppie di utenti: ogni quota di una **Transaction** non attribuita al pagatore genera un debito verso di esso, mentre ogni **Settlement** viene trattato come un debito negativo che cancella il debito corrispondente nella stessa direzione. Da questa lista si ottengono sia il saldo netto complessivo di un utente nel gruppo, sia il dettaglio del saldo verso ciascun altro membro. **StatisticsCalculator** aggrega invece l'importo totale e il numero di transazioni per categoria di spesa, considerando l'importo pieno di ogni **Transaction** indipendentemente da come è ripartito,

e i soli movimenti di tipo spesa: i **Settlement**, essendo rimborsi, sono esclusi dal conteggio.

3.2.5 Strategia di calcolo

Il calcolo del bilancio crea una struttura a coppie ordinate (debitore, creditore) popolata con un solo passaggio su transazioni e pagamenti diretti. Le richieste di saldo netto complessivo e dettaglio verso un singolo membro vengono poi derivate da questa struttura senza ricalcoli separati.

Il punto centrale della strategia è trattare un **Settlement** come un debito *negativo* nella stessa direzione del pagamento (da chi salda a chi riceve), anziché come un'operazione a parte da sottrarre esplicitamente da un debito esistente. In questo modo l'unica funzione `ADDDEBT` basta per entrambi i tipi di movimento: un pagamento diretto tra due utenti che non hanno debiti pregressi tra loro produce semplicemente un debito negativo, mentre se il beneficiario doveva effettivamente qualcosa al pagatore, il debito negativo si compensa con quello esistente nella stessa coppia senza bisogno di un caso particolare.

Algorithm 1 Costruzione del registro dei debiti (**BalanceCalculator.of**)

```
1: procedure COSTRUISCIREGISTRO(transazioni, pagamenti)
2:   saldoNetto  $\leftarrow$  mappa vuota (utente  $\rightarrow$  importo)
3:   debitoACoppie  $\leftarrow$  mappa vuota ((debitore, creditore)  $\rightarrow$  importo)
4:   for all t  $\in$  transazioni do
5:     for all quota  $\in$  t.ripartizione do
6:       if quota.utente  $\neq$  t.pagatore then
7:         importo  $\leftarrow$  t.importo  $\times$  quota.percentuale/100
8:         AGGIUNGIDEBITO(quota.utente, t.pagatore, importo)
9:       end if
10:    end for
11:  end for
12:  for all p  $\in$  pagamenti do
13:    AGGIUNGIDEBITO(p.pagatore, p.beneficiario,  $-p.importo$ )
14:  end for
15:  return (saldoNetto, debitoACoppie)
16: end procedure
17: procedure AGGIUNGIDEBITO(debitore, creditore, importo)
18:   debitoACoppie[debitore, creditore]  $\leftarrow$ 
     debitoACoppie[debitore, creditore] + importo
19:   saldoNetto[creditore]  $\leftarrow$  saldoNetto[creditore] + importo
20:   saldoNetto[debitore]  $\leftarrow$  saldoNetto[debitore] - importo
21: end procedure
```

Le due operazioni di lettura si riducono quindi a semplici accessi alla struttura già calcolata:

- $\text{SALDONETTO}(\textit{utente}) = \textit{saldoNetto}[\textit{utente}]$;
- $\text{SALDOVERSO}(\textit{da}, \textit{a}) = \textit{debitoACoppie}[\textit{da}, \textit{a}] - \textit{debitoACoppie}[\textit{a}, \textit{da}]$, la quota che *da* deve a *a* al netto della direzione opposta.

3.3 Diagrammi di sequenza

Le sequenze seguenti illustrano la creazione di una **Transaction** e di una richiesta di **Balance**. Il pattern mostrato è analogo per tutti gli altri elementi del servizio.

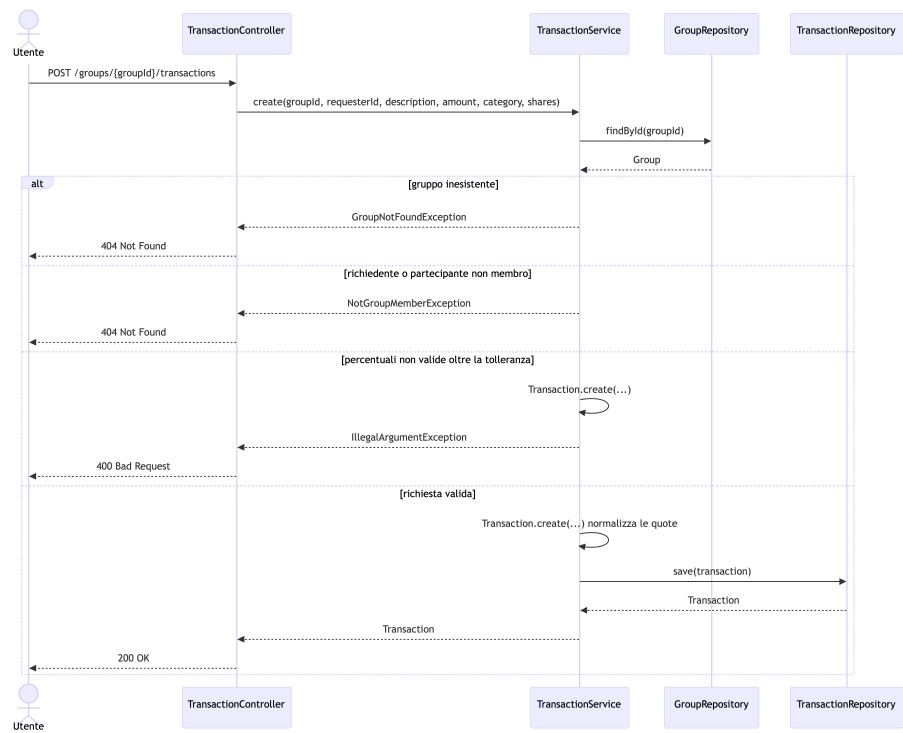


Figura 6: Creazione di una transazione con verifica di appartenenza al gruppo)

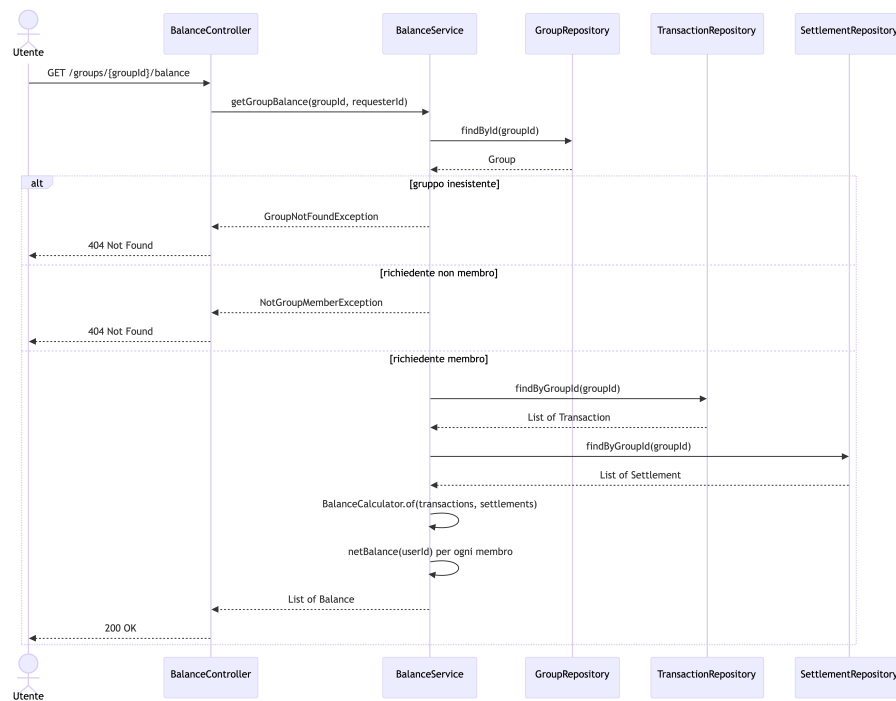


Figura 7: Richiesta del bilancio di un gruppo

3.4 Casi d'uso

Di seguito sono dettagliati i casi d'uso più rappresentativi dell'iterazione; i restanti sono analoghi e sono descritti sinteticamente al termine della sezione.

3.4.1 UC3.1 - Creazione gruppo

- **Attori:** Utente autenticato;
- **Descrizione:** un utente crea un nuovo gruppo di spesa, diventandone automaticamente proprietario;
- **Precondizioni:** l'utente deve possedere un profilo (**UserProfile**) esistente;
- **Postcondizioni:** viene creato un nuovo gruppo con un unico membro (l'utente stesso), in stato **ACCEPTED** e ruolo **OWNER**;
- **Flusso degli eventi principale:**

1. l'utente invia una richiesta di creazione (nome del gruppo) a `POST /groups`;
2. il servizio valida che il nome non sia vuoto e non superi la lunghezza massima consentita;
3. viene creato un nuovo **Group** con l'utente richiedente come unico membro, ruolo **OWNER**;
4. il gruppo creato viene restituito, incluso l'identificativo assegnato.

- **Flussi alternativi:**

- *Nome non valido:* nome vuoto o troppo lungo comportano un errore di validazione (`400 Bad Request`).

3.4.2 UC3.2 - Invito

- **Attori:** Proprietario del gruppo (invitante), utente invitato;
- **Descrizione:** il proprietario di un gruppo invita un altro utente, registrato in *ExpSplit*, a farne parte; l'invito resta in sospeso finché l'invitato non lo accetta o lo rifiuta;
- **Precondizioni:** l'invitante deve essere proprietario del gruppo; l'utente invitato deve possedere un profilo (**UserProfile**) esistente e non essere già membro del gruppo;
- **Postcondizioni:** viene aggiunta una nuova **Membership** in stato **PENDING** per l'utente invitato, con ruolo **MEMBER**;
- **Flusso degli eventi principale:**
 1. il proprietario invia una richiesta di invito (identificativo dell'invitato) a `POST /groups/{groupId}/invitations`;
 2. il servizio verifica che il gruppo esista;
 3. il servizio verifica che l'utente invitato possieda un profilo esistente;
 4. il dominio verifica che il richiedente sia proprietario del gruppo e che l'invitato non sia già membro;
 5. viene aggiunta una nuova **Membership** in stato **PENDING**;
 6. l'utente invitato, con una richiesta separata a `POST /groups/{groupId}/invitations/accept`, accetta l'invito: la **Membership** transita in stato **ACCEPTED**.

- **Flussi alternativi:**

- *Richiedente non proprietario*: l'operazione viene rifiutata (403 Forbidden);
- *Utente invitato senza profilo*: l'operazione viene interrotta (404 Not Found);
- *Utente già membro*: l'operazione viene rifiutata (409 Conflict);
- *Rifiuto dell'invito*: l'invitato può inviare POST `/groups/{groupId}/invitations/reject`, rimuovendo la Membership in sospeso.

3.4.3 UC4.1 - Creazione transazione

- **Attori:** Membro del gruppo;
- **Descrizione:** un membro registra una spesa condivisa, ripartendola tramite percentuali personalizzate tra i membri del gruppo;
- **Precondizioni:** il richiedente deve essere membro del gruppo; ogni utente indicato nella ripartizione deve anch'esso essere membro del gruppo;
- **Postcondizioni:** viene creata una nuova **Transaction**, con il richiedente come pagatore, categoria (facoltativa, predefinita **OTHER**) e ripartizione salvata sommando esattamente a 100;
- **Flusso degli eventi principale:**
 1. il richiedente invia descrizione, importo, categoria facoltativa e ripartizione (percentuale per utente) a POST `/groups/{groupId}/transactions`;
 2. il servizio verifica che il gruppo esista e che il richiedente ne sia membro;
 3. il servizio verifica che ogni utente indicato nella ripartizione sia membro del gruppo;
 4. il dominio verifica che le percentuali non abbiano utenti duplicati e sommino a 100 (con tolleranza di arrotondamento assorbita automaticamente nell'ultima quota);
 5. viene creata e salvata la nuova transazione.
- **Flussi alternativi:**

- *Richiedente o partecipante non membro*: l'operazione viene interrotta (404 Not Found);
- *Percentuali non valide oltre la tolleranza di arrotondamento*: l'operazione viene rifiutata (400 Bad Request).

3.4.4 UC5.1 - Aggiornamento (bilancio di gruppo)

- **Attori:** Membro del gruppo;
- **Descrizione:** un membro richiede il saldo netto, sempre aggiornato, di ciascun membro del gruppo rispetto al registro delle transazioni e dei pagamenti diretti;
- **Precondizioni:** il richiedente deve essere membro del gruppo;
- **Postcondizioni:** nessuna: l'operazione è di sola lettura, il risultato viene ricalcolato ad ogni richiesta e non viene salvato;
- **Flusso degli eventi principale:**
 1. il richiedente invia GET /groups/{groupId}/balance;
 2. il servizio verifica che il gruppo esista e che il richiedente ne sia membro;
 3. il servizio recupera tutte le transazioni e i pagamenti diretti del gruppo;
 4. viene costruito il registro dei debiti e calcolato il saldo netto di ciascun membro;
 5. l'elenco dei saldi, uno per membro, viene restituito.
- **Flussi alternativi:**
 - *Richiedente non membro*: l'operazione viene interrotta (404 Not Found).

3.4.5 Altri casi d'uso

I restanti casi d'uso applicano schemi già descritti sopra (verifica di appartenenza/proprietà del gruppo, poi lettura, modifica o cancellazione) e sono riassunti di seguito:

- **UC2.1 - Modifica:** PATCH /profile aggiorna nome e cognome del profilo dell'utente autenticato;
- **UC2.2 - Eliminazione:** DELETE /profile elimina permanentemente il profilo; una successiva lettura restituisce 404, senza ricreazione implicita;
- **UC3.3 - Abbandono:** DELETE /groups/{groupId}/members/me; se il richiedente è l'unico membro accettato il gruppo viene eliminato, se è il proprietario e restano altri membri il più anziano viene promosso;
- **UC3.4 - Eliminazione:** DELETE /groups/{groupId}, riservata al proprietario (403 altrimenti);
- **UC4.2 - Modifica:** PATCH /groups/{groupId}/transactions/{transactionId}, riservata a chi ha creato la transazione o al proprietario del gruppo, con le stesse verifiche di UC4.1;
- **UC4.3 - Cancellazione:** DELETE /groups/{groupId}/transactions/{transactionId} e l'analogo DELETE .../settlements/{settlementId}, riservate a chi ha creato il movimento o al proprietario del gruppo;
- **UC5.2 - Dettaglio:** GET /groups/{groupId}/balance/me restituisce il saldo netto del richiedente e il dettaglio verso ciascun altro membro con saldo non nullo;
- **UC5.3 - Statistiche:** GET /groups/{groupId}/statistics restituisce, per ogni categoria con almeno una transazione, l'importo totale e il numero di transazioni, ordinati per importo decrescente.

3.5 Modello REST

La tabella seguente riepiloga gli endpoint esposti da *ExpSplit*, tutti protetti da autenticazione JWT e raggiungibili tramite il *Gateway*.

Metodo	Percorso	Caso d'uso
POST	/profile	UC2
GET	/profile	UC2
PATCH	/profile	UC2.1
DELETE	/profile	UC2.2
POST	/groups	UC3.1
GET	/groups/{groupId}	UC3
POST	/groups/{groupId}/invitations	UC3.2
POST	/groups/{groupId}/invitations/accept	UC3.2
POST	/groups/{groupId}/invitations/reject	UC3.2
DELETE	/groups/{groupId}/members/me	UC3.3
DELETE	/groups/{groupId}	UC3.4
POST	/groups/{groupId}/transactions	UC4.1
GET	/groups/{groupId}/transactions	UC4
GET	/groups/{groupId}/transactions/{id}	UC4
PATCH	/groups/{groupId}/transactions/{id}	UC4.2
DELETE	/groups/{groupId}/transactions/{id}	UC4.3
POST	/groups/{groupId}/settlements	UC4.1
GET	/groups/{groupId}/settlements	UC4
GET	/groups/{groupId}/settlements/{id}	UC4
DELETE	/groups/{groupId}/settlements/{id}	UC4.3
GET	/groups/{groupId}/balance	UC5.1
GET	/groups/{groupId}/balance/me	UC5.2
GET	/groups/{groupId}/statistics	UC5.3

Tabella 1: Endpoint REST del servizio ExpSplit

3.6 Testing

Ogni caso d'uso è coperto da due livelli di test:

- **Test unitari** (JUnit): verifica senza persistenza il dominio.
- **Test di integrazione** (MockMvc + JUnit, database H2 in memoria): verificano l'intero flusso HTTP → controller → servizio applicativo → persistenza per ciascun elemento, inclusi i casi di errore.

Allo stato attuale sono stati creati 134 casi di test. Ogni funzionalità è stata inoltre verificata manualmente con Postman con un database PostgreSQL.

✓	✓ Test Results	10 sec 887 ms
>	✓ BalanceIntegrationTest	2 sec 970 ms
>	✓ GroupIntegrationTest	2 sec 457 ms
>	✓ SettlementIntegrationTest	1 sec 465 ms
>	✓ StatisticsIntegrationTest	605 ms
>	✓ TransactionIntegrationTest	1 sec 793 ms
>	✓ UserProfileIntegrationTest	968 ms
>	✓ BalanceCalculatorTest	3 ms
>	✓ GroupNameTest	2 ms
>	✓ GroupTest	9 ms
>	✓ SettlementTest	2 ms
>	✓ StatisticsCalculatorTest	3 ms
>	✓ TransactionTest	4 ms
>	✓ UserProfileTest	1 ms

Figura 8: Risultati test

4 Iterazione 3

4.1 Introduzione

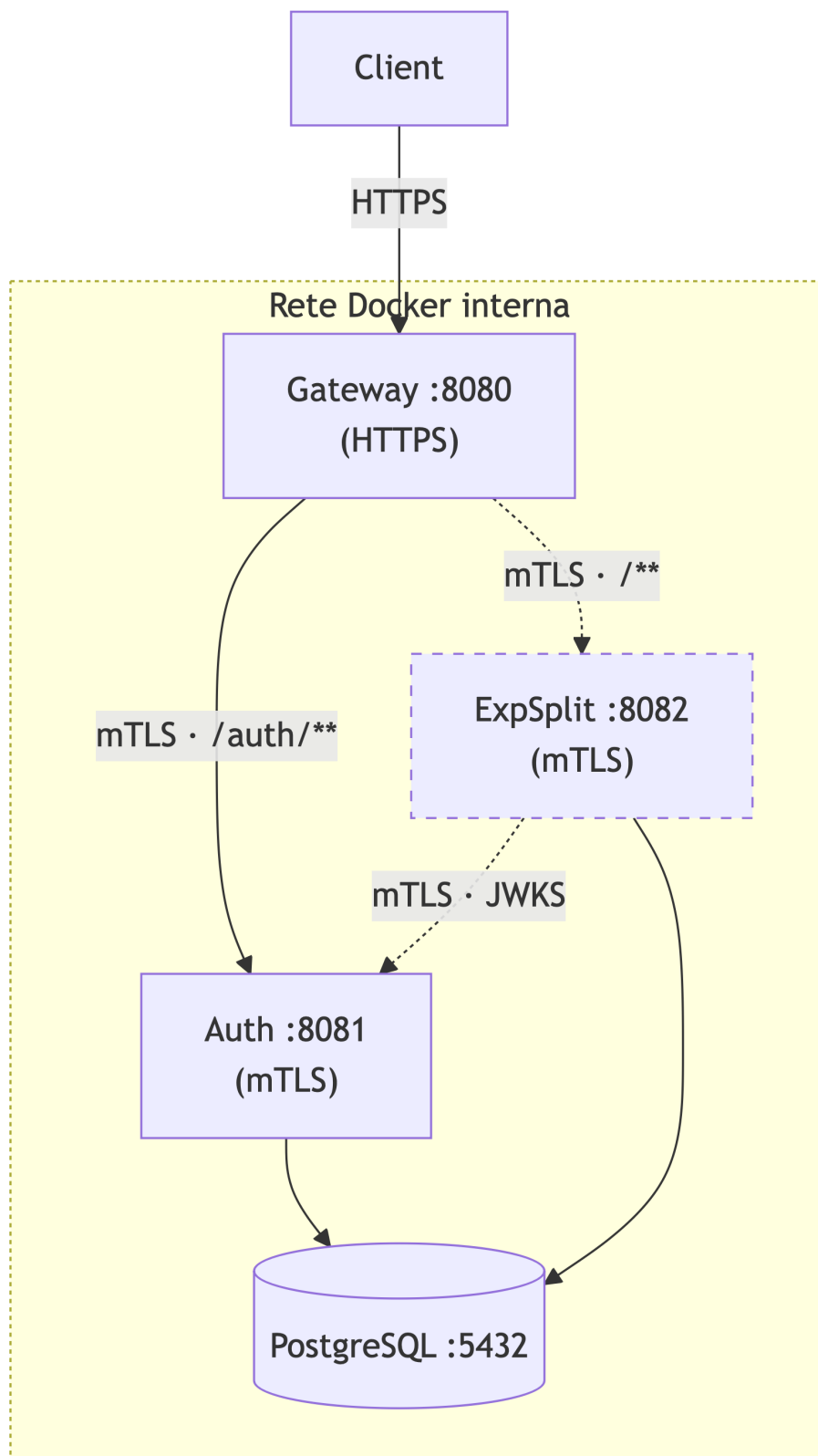
L'iterazione 3 si concentra sul *deployment effettivo* del sistema: containerizzazione dei tre servizi con Docker Compose, cifratura e autenticazione reciproca del traffico interno (*mTLS*).

Si aggiungono inoltre due rifiniture pensate per l'utilizzo da parte di un client. È stato introdotto un endpoint dedicato per recuperare gli inviti in sospeso di un utente: senza sarebbe stato impossibile per il client recuperare l'identificativo del gruppo. Allo stesso modo le risposte REST che coinvolgono altri utenti sono state arricchite con le relative informazioni di profilo, evitando di dover aggiungere ulteriori endpoint e/o richieste aggiuntive da parte del client.

4.2 Deployment con Docker

I tre servizi (*Gateway*, *Auth*, *ExpSplit*) e PostgreSQL vengono orchestrati tramite un unico `docker-compose.yml` alla radice del repository. Le immagini dei tre servizi Spring Boot condividono lo stesso `Dockerfile` multi-stage parametrizzato. L'esecuzione di una build multi-stage permette di ridurre il peso dell'immagine finale creata:

- **Stage 1:** immagine con JDK che esegue la compilazione;
- **Stage 2:** immagine con JRE che permette l'esecuzione del `jar` risultante dello **Stage 1**.



4.2.1 Rete

Tutti i servizi comunicano sulla rete creata con Docker Compose. L'unico punto di accesso è il *Gateway* che espone la porta 8080 via HTTPS. *PostgreSQL*, *Auth* ed *ExpSplit* non sono raggiungibili direttamente dall'esterno.

4.2.2 Configurazione e inizializzazione

Per l'inizializzazione del database è necessario avviare uno script che crea i ruoli `expsplit_auth` ed `expsplit_app` con permessi limitati al proprio schema. Le password e i percorsi per i servizi sono impostabili dalle variabili d'ambiente (configurabili nel file `.env`).

4.3 Sicurezza

A differenza delle iterazioni precedenti dove i servizi comunicavano in HTTP in questa iterazione ogni canale di comunicazione è cifrato con distinzione tra il traffico interno e il traffico pubblico.

mTLS tra i servizi interni: Le comunicazioni tra *Gateway*, *Auth* ed *Exp-Split* richiedono *mutual TLS*: ciascun servizio espone un certificato server e richiede, a sua volta, un certificato client valido a chi lo contatta. Ogni servizio quindi dimostra la propria identità con un certificato firmato dalla stessa CA.

HTTPS. Il *Gateway* utilizza invece TLS *server-side* tradizionale: il servizio espone un certificato senza richiesta di un certificato da parte del client.

Certificati. Tutti i certificati sono autofirmati da una CA interna generata da uno script che crea la CA, un truststore condiviso (contenente solo il certificato della CA) e, per ciascun servizio, una coppia di chiavi con relativo certificato. I certificati vengono esportati in formato PKCS12.

4.4 Nuovo caso d'uso

4.4.1 UC3.5 - Elenco inviti in sospeso

- **Attori:** Utente autenticato;
- **Descrizione:** un utente recupera l'elenco di tutti gli inviti a gruppi ancora in sospeso a lui rivolti, senza dover conoscere in anticipo l'identificativo dei gruppi coinvolti;

- **Precondizioni:** nessuna oltre all'autenticazione;
- **Postcondizioni:** nessuna: l'operazione è di sola lettura;
- **Flusso degli eventi principale:**
 1. l'utente invia `GET /invitations`;
 2. il servizio recupera tutti i gruppi in cui l'utente ha una `Membership` in stato `PENDING`;
 3. per ciascun gruppo trovato viene restituito l'identificativo, il nome del gruppo e la data dell'invito;
 4. l'utente prosegue con gli endpoint già esistenti di UC3.2 (`POST /groups/{groupId}/invitations/accept` o `.../reject`) per elaborare la risposta all'invito.
- **Flussi alternativi:**
 - *Nessun invito in sospeso:* viene restituita una lista vuota (200 OK).

Il nuovo endpoint si aggiunge alla tabella 1:

Metodo	Percorso	Caso d'uso
GET	<code>/invitations</code>	UC3.5

Tabella 2: Nuovo endpoint: invitations

4.5 Arricchimento delle risposte REST

Le risposte REST che referenziavano un utente esponevano esclusivamente il relativo `UserProfileId`, delegando a un client applicativo il compito di risolverlo separatamente. In vista di un utilizzo reale da parte di un'applicazione e per ridurre il numero di richieste, ogni riferimento a un utente è stato sostituito con l'oggetto profilo completo (`id`, `email`, `firstName`, `lastName`), lo stesso restituito da `GET /profile`.

L'informazione viene aggiunta lato server con un'unica interrogazione per risposta, anche per le liste, evitando una richiesta separata per ciascun riferimento. Se un profilo è nel frattempo stato eliminato, il rispettivo campo viene restituito semplicemente a `null` senza generare un'eccezione.

4.6 Testing

I casi di test sono stati estesi per contemplare anche il nuovo endpoint. Il nuovo numero di test è 137 divisi tra test unitari e test di integrazione. I casi di test utilizzano il profilo `nomtls` e non sono quindi impattati dalle modifiche relative alla sicurezza apportate. La verifica di tutto il sistema con HTTPS, mTLS e Docker è stata eseguita manualmente con `Postman`.

5 Iterazione 4

5.1 Introduzione

L'iterazione 4 introduce un'applicazione iOS sviluppata in *SwiftUI*, attraverso cui si testano i casi d'uso realizzati nelle iterazioni precedenti. Il client comunica con i servizi tramite le API REST esposte dal *Gateway* e implementa le funzionalità di autenticazione, gestione dei gruppi e condivisione delle spese.

L'obiettivo dell'iterazione è lo sviluppo di un client con tecnologia diversa e la verifica delle funzionalità proposte.

5.2 Ruolo nell'architettura esagonale

L'applicazione è un *client esterno* che comunica esclusivamente tramite richieste HTTP con i controller REST con scambio di dati in formato JSON. Il client non ha accesso o conoscenza della struttura del servizio e del database.

5.3 Organizzazione del client

Il codice iOS è organizzato come di seguito:

- **Viste SwiftUI:** presentano i dati, raccolgono gli input e gestiscono navigazione, stato di caricamento e messaggi di errore;
- **Moduli API:** definiscono i dati scambiati con il servizio e le operazioni HTTP per le diverse funzionalità, utilizzando chiamate asincrone;
- **APIClient:** centralizza l'indirizzo del Gateway, la sessione di rete e la configurazione della decodifica JSON;
- **AuthStorage:** conserva il token JWT nel *Keychain*. Il token viene inviato con le richieste e rimosso localmente al logout.

Il client esegue controlli sui campi e facilita la composizione delle richieste, ad esempio distribuendo le percentuali di una spesa tra i partecipanti selezionati. La verifica delle autorizzazioni e della validità delle operazioni rimane comunque responsabilità del servizio.

Il client è stato volutamente progettato per essere semplice e utilizzato come strumento di verifica delle funzionalità - non è stato adottato uno stile architetturale tipico delle applicazioni iOS.

5.4 Funzionalità

L'interfaccia permette registrazione e accesso, compilazione e modifica del profilo, creazione e consultazione dei gruppi, gestione degli inviti e uscita da un gruppo. Nel dettaglio di un gruppo sono disponibili spese, pagamenti diretti, membri, bilanci e statistiche per categoria. Queste operazioni sono descritte nel dettaglio nei casi d'uso nelle iterazioni precedenti.

5.5 Verifica e limiti

La verifica dell'integrazione deve coprire almeno un flusso completo: accesso, creazione di un gruppo, invito e accettazione da parte di un secondo utente, inserimento di una spesa e consultazione dei bilanci. Altri casi d'uso secondari o interazioni più complesse sono state omesse per semplicità.